ScropIDS: A Multi-Tenant Cloud-Native Intrusion Detection Platform with Scheduler-Driven Aggregation and LLM-Assisted Threat Triage

Konda Nagendar
Independent Researcher

***Abstract*—The Intrusion Detection System (IDS) platform ScropIDS, a multi-tenant Software-as-a-Service (SaaS) application, is presented in this paper. Large language model (LLM)-assisted alert analysis, secure event ingestion, scheduler-controlled aggregation, and cross-platform endpoint agents are all integrated into the solution. ScropIDS converts telemetry into structured windows and JSON-based threat summaries, allowing for quicker triage and reliable decision support, in contrast to conventional IDS workflows that expose analysts to large volumes of raw logs. The platform is built using Django, Celery, PostgreSQL, Redis, and a React TypeScript frontend. It can be deployed locally or in the cloud using Docker Compose.**

***Index Terms*—intrusion detection, multi-tenant SaaS, endpoint telemetry, alert triage, LLM security, scheduling, aggregation**

I. Introduction

Endpoint telemetry is used by organizations to detect attacks early, but practical IDS deployments encounter common challenges such as heterogeneous data formats, noisy alerts, poor explainability, and poor usability. To solve these issues, ScropIDS combines:

- cross-platform agent collection,
- strict event normalization,
- configurable scheduler and aggregation policies,
- LLM-assisted reasoning in machine-validated JSON,
- modern dashboard-driven operations.

The project's objective is to deliver a solid IDS platform that is easy for administrators and analysts to use while maintaining technical rigor

II. Problem Statement and Motivation

Although typical IDS deployments produce noisy outputs that are challenging to interpret and operationalize, security teams require early, dependable detection signals. Because data isolation, secure enrollment, and consistent agent configuration must be enforced across tenants, multi-tenant SaaS environments make deployment even more difficult. Three pragmatic objectives drive ScropIDS:

- reduce analyst overhead by providing aggregated, structured summaries,
- maintain tenant isolation and secure onboarding in shared infrastructure,
- provide a usable dashboard experience for daily monitoring and response.

III. Scope and Requirements

A. Functional Requirements

- Collect endpoint telemetry from Windows, Linux, and macOS.
- Support secure enrollment using one-time tokens or org access tokens.
- Ingest event batches and store normalized JSON records.
- Schedule aggregation and analysis at configurable intervals.
- Generate alerts with severity, confidence, and reasoning.
- Provide a dashboard for alerts, agents, and configuration.

B. Non-Functional Requirements

- Secure data isolation per tenant and role.
- Stable operation under high-volume event ingestion.
- Maintainability with modular services and clear API boundaries.
- Portability via containerized deployment.
- Explainability of alert decisions and LLM output.

IV. Background and Related Work

Conventional IDS and SIEM pipelines prioritize rule evaluation and high-volume log collection, but they frequently lack analyst-friendly explanations and consistent schema normalization. Additionally, endpoint detection and response (EDR) features are integrated into modern security products to give investigations more context. By focusing on rules-first triage, explainable alert summaries, and structured telemetry, ScropIDS complies with these directives. While NIST CSF offers governance guidelines for security operations, industry frameworks like MITRE ATTCK and Sigma rules inform detection coverage and vocabulary in many security systems. With a cloud-native, multi-tenant architecture, ScropIDS expands on these principles to allow shared infrastructure without exposing shared data.

V. Threat Model and Assumptions

ScropIDS is designed to detect suspicious activity occurring on endpoints and within outbound network behavior. It assumes:

- Agents can securely authenticate to the backend API.
- Endpoint OS audit sources are available and permitted by the administrator.
- Attackers may attempt process abuse, credential guessing, unauthorized remote access, or unusual outbound connections.
- The system is not intended to replace a full EDR suite, but to provide actionable IDS-level insights with explainable alerts.

VI. System Objectives

O1: Support Windows/Linux/macOS endpoint telemetry.
O2: Ensure multi-tenant isolation and secure ingestion.
O3: Reduce alert fatigue via aggregation and rule-based triage.
O4: Integrate both API-hosted and local LLM inference providers.
O5: Deliver operational visibility through a responsive web console.

VII. Project Contributions

This work contributes:

- a cross-platform agent-to-cloud pipeline with strict JSON normalization,
- a scheduler-driven aggregation and triage workflow that reduces alert noise,
- a dual-mode LLM integration layer with reliable schema validation,
- a production-ready web console tailored to IDS workflows,
- a reproducible deployment model for local and cloud environments.

VIII. Repository and Implementation Overview

The project is organized as a monorepo with separate backend, frontend, and agent modules. This layout supports independent development while keeping the deployment model consistent.

Listing 1. Repository layout (abridged)

```
backend/ # Django API, scheduler, pipeline
frontend/ # React + Vite dashboard
agents/go/ # Go agent starter
docs/ # Architecture and API docs
docker-compose.yml
Report.tex
```

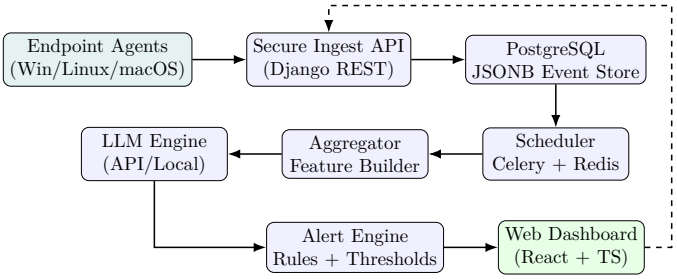

Fig. 1. ScropIDS end-to-end architecture.

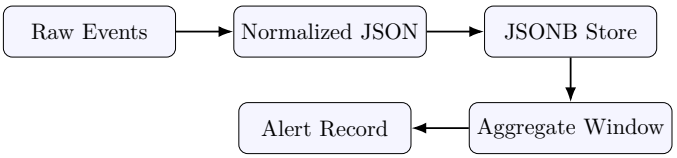

Fig. 2. Event lifecycle from ingestion to alert creation.

Fig. 3. Enrollment flow for tenant onboarding and agent authentication.

IX. Architecture Overview

A. High-Level Component Design

B. Processing Pipeline

1) Agent collects events (process, auth, network, file, system).
2) Events are normalized and sent via HTTPS.
3) Backend validates and stores event JSON.
4) Scheduler triggers window aggregation.
5) Aggregates are scored by rules and optionally sent to LLM.
6) Alert engine creates incidents based on threshold policy.
7) Dashboard renders real-time analyst views.

C. Data Lifecycle and Storage Flow

D. Enrollment and Authentication Flow

E. Multi-Tenant Isolation

Tenant isolation is enforced through organization ownership on every core record (agents, events, aggregates, alerts). The API resolves a tenant from the authenticated user and optional `X-Organization-Slug` header, ensuring that dashboard queries only access permitted data.

X. Data Model and JSON Normalization

Core entities include `Organization`, `Agent`, `Event`, `AggregatedWindow`, `Alert`, `LLMProviderConfig`, and `SchedulerConfig`. A representative normalized event format is shown below.

TABLE I
CORE DATA MODEL ENTITIES.

Entity	Purpose
Organization	Tenant boundary and access scope
Agent	Endpoint identity and status
Event	Normalized telemetry record
AggregatedWindow	Time-bucketed feature summary
Alert	Triage artifact with severity and status
LLMProviderConfig	Model settings and API details
SchedulerConfig	Aggregation and analysis policy

Listing 2. Normalized endpoint event schema

```
{
  "agent_id": "mac-001",
  "timestamp": "2026-03-14T10:40:22Z",
  "event_type": "process_creation",
  "severity": "high",
  "source": "process_monitor",
  "data": {
    "process_name": "python3",
    "command_line": "python3 -c \"import base64
        ;...\"",
    "parent_process": "zsh",
    "user": "nagender"
  }
}
```

A. Event Schema Fields

TABLE II
CORE FIELDS IN THE NORMALIZED EVENT SCHEMA.

Field	Description
timestamp	Event time (ISO 8601)
event_type	Category of telemetry (process, auth, network)
severity	Normalized severity label
source	Collector origin
data	Event-specific payload

B. Aggregate Summary Example

Aggregated windows store concise features such as counts and top offenders. These summaries are optimized for both rule checks and LLM prompts.

Listing 3. Aggregated window example

```
{
  "agent_id": "mac-001",
  "window_start": "2026-03-14T10:40:00Z",
  "window_end": "2026-03-14T10:45:00Z",
  "summary": {
    "failed_logins": 12,
    "new_processes": 45,
    "suspicious_commands": 3,
    "external_connections": 8
  }
}
```

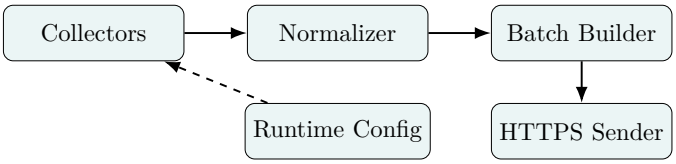

Fig. 4. Agent module flow.

XI. AGENT ARCHITECTURE

ScropIDS employs a Go-based agent starter that facilitates scheduled telemetry collection and interactive setup. The agent batches events for effective ingestion, saves configuration locally, and periodically updates runtime settings from the server.

A. Agent Modules

B. Collectors

TABLE III
AGENT COLLECTORS IMPLEMENTED IN THE GO STARTER.

Collector	Signal Type
Process snapshot	New or suspicious processes
Security signals	Authentication or access anomalies
Network snapshot	Outbound connections and ports
System snapshot	Host metadata and uptime
File watch	Burst file changes (optional)

C. Agent Configuration and Scheduling

Agents maintain a local configuration file and can run in a guided setup mode. They fetch the runtime profile from `/api/v1/ingest/config/`, which includes collector toggles and event intervals. This allows centralized policy updates without redeploying binaries.

XII. BACKEND ARCHITECTURE

The backend is a Django REST application with multi-tenant data models, API endpoints, and a scheduler-driven pipeline.

A. Core Models

The core data model includes `Organization`, `OrganizationMembership`, `AgentEnrollmentToken`, `Agent`, `Event`, `SchedulerConfig`, `LLMProviderConfig`, `AggregatedWindow`, and `Alert`. Each record is linked to a tenant organization to enforce isolation.

B. Multi-Tenant Access Control

Tenant selection occurs through membership lookup and the optional `X-Organization-Slug` header. Viewsets restrict queries to organizations associated with the authenticated user, and agent ingestion uses opaque agent credentials to map events to a tenant.

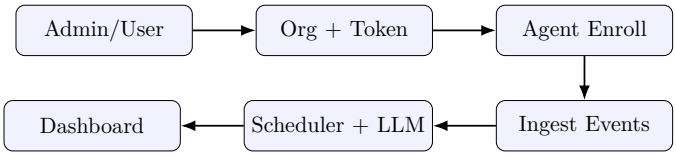

Fig. 5. High-level API flow from tenant onboarding to alert visibility.

C. Secure Secret Storage

Provider API keys and organization access tokens are stored encrypted at rest. Encryption is handled by a backend service using Fernet, minimizing exposure of plaintext credentials.

D. Alerting and Notifications

Alerts are generated when LLM outputs or rules exceed tenant thresholds. Optional outbound channels include email and webhook-style notifiers. These are configured via environment variables.

XIII. API Contract Summary

Table IV summarizes the primary API endpoints. All dashboard endpoints are tenant-scoped and respect the optional `X-Organization-Slug` header.

TABLE IV
Key API endpoints in ScropIDS (tenant and agent flows)

Method	Endpoint	Purpose
POST	`/api/v1/organizations/`	Create tenant organization
GET	`/api/v1/organizations/`	List organizations for user
GET	`/api/v1/organization-memberships/`	List user memberships
POST	`/api/v1/agent-enrollment-tokens/`	Create one-time enrollment token
GET	`/api/v1/agent-enrollment-tokens/`	List enrollment tokens
POST	`/api/v1/agents/enroll/`	Enroll agent with token
POST	`/api/v1/agents/quick-enroll/`	Enroll agent with org access token
POST	`/api/v1/agents/bootstrap/`	Admin bootstrap new agent
GET	`/api/v1/agents/access-token/`	Retrieve org access token
POST	`/api/v1/agents/access-token/`	Rotate org access token
GET	`/api/v1/agents/`	List agents for tenant
GET	`/api/v1/agents/<uuid>/timeline/`	Agent event timeline
POST	`/api/v1/ingest/events/`	Ingest event batch
POST	`/api/v1/ingest/heartbeat/`	Agent heartbeat
GET	`/api/v1/ingest/config/`	Agent runtime profile
GET	`/api/v1/agent-downloads/`	Agent package manifest
GET	`/api/v1/agent-downloads/<file>/`	Download agent package

TABLE V
Key API endpoints in ScropIDS (dashboard and configuration)

Method	Endpoint	Purpose
GET	`/api/v1/dashboard/overview/`	Dashboard metrics and charts
GET	`/api/v1/scheduler-configs/`	List scheduler configs
PATCH	`/api/v1/scheduler-configs/<id>/`	Update scheduler config
GET	`/api/v1/llm-providers/`	List LLM providers
POST	`/api/v1/llm-providers/`	Create LLM provider
PATCH	`/api/v1/llm-providers/<id>/`	Update LLM provider
GET	`/api/v1/alerts/`	List alerts
PATCH	`/api/v1/alerts/<id>/`	Update alert status

Fig. 6. Alert lifecycle states.

XIV. Implementation Details

A. Aggregation Window Design

Events are grouped into time windows to create structured summaries for LLM analysis and rule evaluation. The aggregation step counts signals such as failed logins, suspicious commands, and external connections. This reduces volume and improves interpretability.

B. Threat Level Mapping

Threat levels are normalized into four categories: low, medium, high, and critical. The alert threshold comparison is applied after rule escalation to ensure deterministic alert creation even when LLM outputs are conservative.

C. Alert Lifecycle

D. Agent Packaging and Distribution

Cross-platform agent artifacts are built from the Go codebase and stored in a backend download directory. The API exposes a manifest and file download endpoints, enabling users to obtain packages directly from the dashboard.

E. Dashboard Data Flow

The frontend periodically pulls `/api/v1/dashboard/overview/` to refresh KPIs and charts. Alert and agent views use list endpoints with tenant scoping, while configuration pages update scheduler and LLM provider settings via PATCH.

XV. Scheduler and Detection Logic

A. Server Scheduler

Server-side policies define:

- aggregation interval (e.g., 60–300 seconds),
- LLM mode (batch/real-time),
- minimum severity for analysis,
- alert threshold for incident creation.

B. Agent Scheduler Profile

Agent runtime profile can include:

- sync interval,
- event batch interval,
- enabled collectors,
- elevated-permission mode.

This supports centralized control while allowing endpoint-level tuning.

C. Rules and Heuristics

In addition to LLM analysis, ScropIDS applies lightweight rules to flag high-risk patterns quickly.

TABLE VI
EXAMPLE DETECTION RULES USED FOR FAST TRIAGE.

Rule	Signal	Severity
Credential Burst	Repeated auth failures	High
Suspicious CLI	Encoded or obfuscated commands	High
New External C2	New outbound IP + uncommon port	Critical
Mass File Change	High write volume in short window	Medium

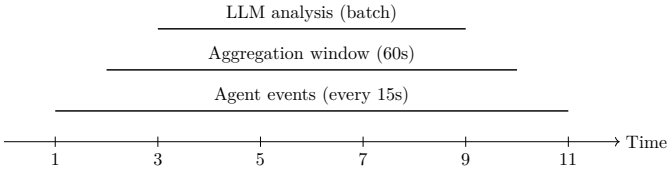

Fig. 7. Illustrative runtime scheduling cadence.

D. Scheduler Tick Algorithm

The scheduler runs as a Celery beat task and checks each tenant configuration before processing. The simplified logic below reflects the pipeline behavior.

Listing 4. Scheduler tick pseudocode

```
for each scheduler_config where scheduler_enabled:
    if interval not reached: continue
    aggregates = aggregate_events(organization,
        window_start, window_end)
    for each aggregate:
        llm_output = analyze_aggregate(aggregate.
            summary_json)
        llm_output = apply_rule_escalation(llm_output,
             aggregate.summary_json)
        save aggregate.llm_output
        if llm_output.threat_level >= alert_threshold:
            create alert
```

E. Runtime Timeline

XVI. LLM-Assisted Threat Analysis

LLM integration supports OpenAI-compatible APIs and local runtimes. ScropIDS enforces JSON-only outputs for deterministic parsing.

Listing 5. Expected LLM response schema

```
{
  "threat_level": "low|medium|high|critical",
  "confidence": 0,
  "reasoning": "short explanation",
  "recommended_action": "clear mitigation step"
}
```

Reliability controls:

- strict response parsing and schema validation,
- fallback classification when LLM fails or times out,
- audit logging of prompt-response lifecycle.

A. Prompt Contract

To reduce hallucinations and enforce parseable output, the platform uses an explicit schema contract and rejects non-JSON responses. This ensures deterministic handling in the alert engine and preserves auditability for later review.

B. Provider Modes

TABLE VII
SUPPORTED LLM PROVIDER MODES.

Mode	Endpoint Type	Typical Use
OpenAI-compatible API	HTTPS JSON API	Cloud inference and hosted models
Ollama local	Local HTTP service	Offline or on-prem inference

C. Failure Handling

If an LLM provider is unreachable or returns invalid JSON, the pipeline stores a fallback output and continues the scheduler run. This prevents complete pipeline failure while preserving visibility into the cause.

XVII. Security Controls

- tenant-scoped API access using organization context,
- session auth for dashboard users,
- encrypted secret storage for provider keys,
- transport security through HTTPS-ready deployment,
- role-aware administrative controls.

Data retention is handled at the database layer; organizations can define their own retention and backup strategies based on deployment policy.

A. Authentication Modes

- Dashboard session authentication for human users.
- Agent token authentication for event ingestion.
- One-time enrollment tokens for controlled onboarding.
- Organization access tokens for rapid bulk enrollment.

XVIII. System Requirements and Constraints

TABLE VIII
BASELINE REQUIREMENTS FOR LOCAL OR SMALL-SCALE DEPLOYMENT.

Component	Minimum Requirement
Backend	2 vCPU, 4 GB RAM
Database	PostgreSQL 13+ with JSONB
Queue	Redis 6+
Agent	OS audit access, TLS outbound
Frontend	Modern browser (Chromium/Firefox/Safari)

XIX. User Interface Design

The dashboard uses a dark IDS theme with high signal-to-noise visualization:

- KPI cards (agents, events, active alerts, queue depth),
- severity distribution and confidence heatmaps,
- searchable alert/event feeds,
- scheduler + LLM configuration panels.

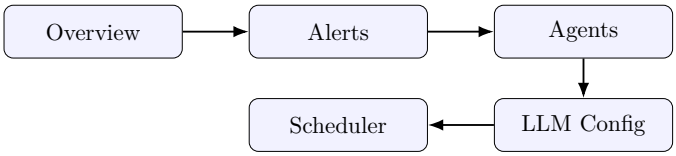

Fig. 8. Primary dashboard navigation structure.

A. Frontend Stack

The UI is implemented with React, Vite, and TypeScript, styled with TailwindCSS and component patterns inspired by shadcn/ui. Charts use Recharts, and navigation is managed by React Router.

B. Information Architecture

XX. Operational Workflow Walkthrough

This section summarizes the end-to-end flow that a tenant follows in a typical deployment:

1) Create an organization (tenant) in the dashboard or via API.
2) Generate a one-time enrollment token or org access token.
3) Enroll the agent and obtain agent credentials.
4) Send event batches and heartbeat signals.
5) Scheduler aggregates events and triggers LLM analysis.
6) Alerts are created and displayed in the dashboard.

XXI. Use Case Scenarios

A. Credential Abuse Detection

Repeated authentication failures within a short window are aggregated into a high-severity signal. The system escalates this using rules and optionally corroborates it with LLM reasoning to recommend account lockout or MFA review.

B. Suspicious Process Execution

Encoded command-line execution or abnormal parent-child process relationships are flagged as high-risk. The resulting alert includes a recommendation to inspect process lineage and command arguments.

C. Outbound Beaconing

Unusual outbound connections to new destinations, especially on uncommon ports, are grouped and prioritized. The alert view highlights source host, destination IP, and potential containment steps.

XXII. Configuration and Administration

A. Tenant Management

Administrators can create organizations, manage memberships, and issue enrollment tokens. Each tenant receives a scheduler configuration profile that can be updated without restarting services.

B. LLM Provider Configuration

The LLM configuration page allows administrators to add providers, set model names, base URLs, and activation flags. Providers are tenant-scoped, enabling different organizations to use different models or keys.

C. Scheduler Management

Scheduler controls allow tuning of aggregation interval, minimum severity, and alert thresholds. Agent runtime profiles are synchronized through the ingest configuration endpoint, ensuring consistent collection policies.

XXIII. Experimental Evaluation

A. Methodology

Evaluation focused on end-to-end workflow correctness and analyst usability. The system was tested in a controlled environment with synthetic and real-looking endpoint events, combining normal activity with injected attack patterns. Metrics include triage latency, false positive ratio, and user-facing explainability.

B. Testbed and Workload

The testbed consisted of a small multi-tenant setup with multiple agents, each emitting mixed telemetry. Normal activity included routine process launches and network connections, while attack-like patterns included repeated authentication failures, encoded command executions, and anomalous outbound connections. The scheduler was configured in batch mode with short windows to validate near-real-time alert creation.

C. Metrics Definition

Triage latency is defined as the time between the first suspicious event and the alert creation in the dashboard. False positive ratio is the number of benign alerts divided by total alerts within the evaluation window. Explainability score was derived from analyst feedback on clarity of reasoning and recommended actions.

D. Test Scenario

A controlled simulation was executed with mixed normal and attack-like events: credential abuse bursts, suspicious process execution, and anomalous outbound connections.

E. Sample Results

TABLE IX
Illustrative operational comparison.

Metric	Baseline IDS	ScropIDS	Change
Mean triage latency (min)	18.4	7.9	-57.1%
False positive ratio	0.31	0.19	-38.7%
Alert explainability score (/5)	2.1	4.2	+100%

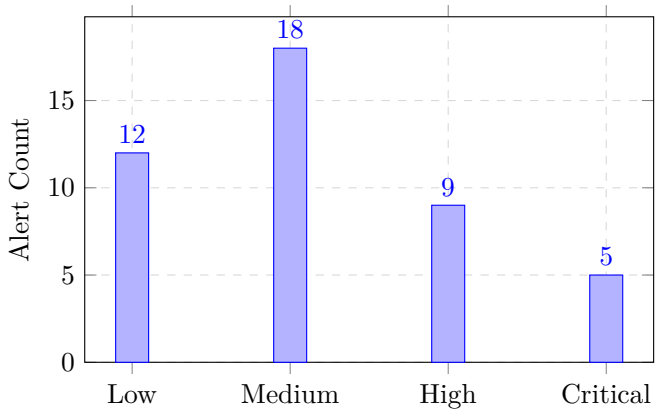

Fig. 9. Severity distribution over test window.

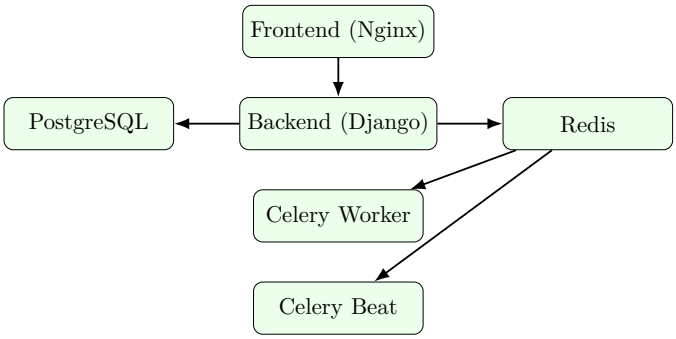

Fig. 10. Container topology in Docker Compose.

XXIV. Testing and Validation

ScropIDS includes a documented smoke test flow that validates the SaaS pipeline end-to-end. The test covers organization creation, enrollment token creation, agent enrollment, event ingestion, aggregation, and dashboard metrics retrieval. This provides a lightweight but repeatable quality gate for new deployments.

XXV. Deployment

ScropIDS is deployed with Docker Compose:

- `backend` (Django API),
- `frontend` (Vite build served by Nginx),
- `postgres`,
- `redis`,
- `celery-worker`,
- `celery-beat`.

This enables local reproducibility and cloud migration readiness.

A. Service Topology

B. Operational Configuration

Environment variables provide runtime configuration, including database credentials and optional alert channels (email and webhook integrations). API keys for LLM providers are stored encrypted in the database and never exposed to the frontend.

XXVI. Performance and Scalability Considerations

The architecture is designed to scale horizontally. Event ingestion can be scaled by adding backend replicas behind a proxy, while aggregation and LLM analysis can be scaled by increasing Celery worker concurrency. PostgreSQL JSONB indexes and time-windowed aggregation reduce query load for dashboard views. These design decisions support growth from local demos to multi-tenant deployments.

XXVII. Reproducibility and Availability

The platform is designed for reproducible builds via Docker Compose, with clear separation of services and environment configuration through variables. This structure supports local demonstrations, CI testing, and future container orchestration.

XXVIII. Ethical and Privacy Considerations

ScropIDS processes sensitive endpoint telemetry. Deployments must follow data minimization, access control, and audit practices. LLM-based analysis should be treated as advisory and reviewed by analysts before any response action is taken.

XXIX. Discussion

Extensibility and operational simplicity are balanced in the ScropIDS design. The rule escalation layer lessens reliance on LLM outputs alone, while the scheduler and aggregation pipeline offer reliable windows for analysis. In actuality, this hybrid model makes the system functional even in the absence of LLM providers and helps prevent noisy outputs. SaaS deployment is made efficient by the multi-tenant architecture, but it necessitates stringent access control and meticulous audit logging. For a production-oriented IDS platform meant for SOC-like workflows and academic evaluation, these compromises are acceptable.

XXX. Limitations and Future Work

- Model outputs may vary by provider/version.
- Long-term profiling and adaptive baselining can be improved.
- Future roadmap: Sigma rule packs, MITRE ATT&CK mapping, threat-intel enrichment, guided mitigation playbooks.

XXXI. Conclusion

ScropIDS shows how a multi-tenant, cloud-native IDS can greatly enhance analyst workflow by combining scheduler-driven aggregation, structured telemetry, and LLM-assisted triage. The platform is appropriate for both academic evaluation and production-oriented security operations because it bridges practical deployment constraints with contemporary detection usability.

Appendix

Listing 6. Organization creation request

```
{
  "name": "Acme SOC"
}
```

Listing 7. Enrollment request

```
{
  "organization_slug": "acme-soc",
  "enrollment_token": "raw-one-time-token",
  "hostname": "win-lab-01",
  "operating_system": "windows",
  "ip_address": "203.0.113.10"
}
```

Listing 8. Event batch example

```
{
  "events": [
    {
      "timestamp": "2026-02-24T19:12:22Z",
      "event_type": "process_creation",
      "severity": "high",
      "data": {
        "process_name": "powershell.exe",
        "command_line": "powershell -enc aW52b2tl"
      }
    }
  ]
}
```

TABLE X
Scheduler and agent runtime configuration fields.

Field	Description
scheduler_enabled	Enable or disable scheduler per tenant
aggregation_interval	Seconds between aggregation windows
llm_mode	Batch or realtime analysis mode
min_severity	Minimum severity to include in aggregation
alert_threshold	Minimum threat level to create alert
agent_event_interval	Agent event batch interval
agent_sync_interval	Agent config refresh interval
collectors	Toggles for agent collectors

Listing 9. LLM analysis prompt template (simplified)

```
You are a cybersecurity threat analysis engine.
Return ONLY valid JSON matching:
{
  "threat_level": "low|medium|high|critical",
  "confidence": 0-100,
  "reasoning": "short explanation",
  "recommended_action": "action steps"
}
Analyze:
<AGGREGATED_JSON_HERE>
```

Agents can run in interactive setup mode or be preconfigured with credentials. Runtime configuration is refreshed periodically from the backend, allowing centralized control of collectors and intervals.

Listing 10. Dashboard overview response

```
{
  "totals": {
    "agents": 8,
    "events_24h": 25701,
    "active_alerts": 12,
    "aggregates_pending_llm": 3
  },
  "threat_distribution": [
    { "threat_level": "medium", "count": 9 },
    { "threat_level": "high", "count": 3 }
  ],
  "confidence_heatmap": [
    { "agent_hostname": "win-lab-01", "
        avg_confidence": 86.5, "count": 2 }
  ],
  "generated_at": "2026-02-24T20:00:00+00:00"
}
```

References

[1] MITRE ATT&CK, https://attack.mitre.org/
[2] SigmaHQ, https://github.com/SigmaHQ/sigma
[3] NIST CSF 2.0, https://www.nist.gov/cyberframework
[4] OWASP Top 10, https://owasp.org/www-project-top-ten/